LABORATUVAR 5 · EFO MAKİNA · BESLEME BANDI & SARSAK

SBOM / CVE / Patch

Çıktı: Vulnerability Decision

AD SOYAD	GRUP	EVİDENCE PACK	MAKİNE
Yağız Enes Doğan	Gündüz - B	EP 09 · 10 · 11	Efo Makina · Besleme Bandı & Sarsak (Titreşimli Konveyör)

Bağımsız akademik çalışmadır; Efo Makina ile bağlantısı yoktur. Analizin birçok kısmı varsayıma dayanır ve gerçek ürünün emniyet durumu hakkında bir iddia değildir.

Kavram: SBOM = yazılım malzeme listesi · **CVE** = açığın kayıt numarası · **CVSS** = ciddiyet (0–10). Bir CVE sizi etkiler mi? Üç şart: **bileşen var**, **sürüm etkileniyor**, **açık ulaşılabilir**.

Efo bağlamı: Efo bir SBOM yayınlamıyor; PLC, HMI ve invertör markaları da bilinmiyor (V2, V3). SBOM bu yüzden LAB 2'deki varlık listesinden kuruldu; sürümler varsayımdır (V16). Föydeki kayıt (Component X 2.4, CVSS 9,8) olduğu gibi kullanıldı; etkilenen sürüm aralığı föyde olmadığı için "2.5 öncesi etkilenir" kabul edildi (V17). Karşılaştırma için iki gerçek kayıt eklendi: S7-1200 için **CVE-2024-47100** ve S7-1200'ü **etkilemeyen** SSA-688146 (Makine Profili D5, D6). Ulaşılabilirlik, LAB 4'teki saldırı grafından hesaplandı.

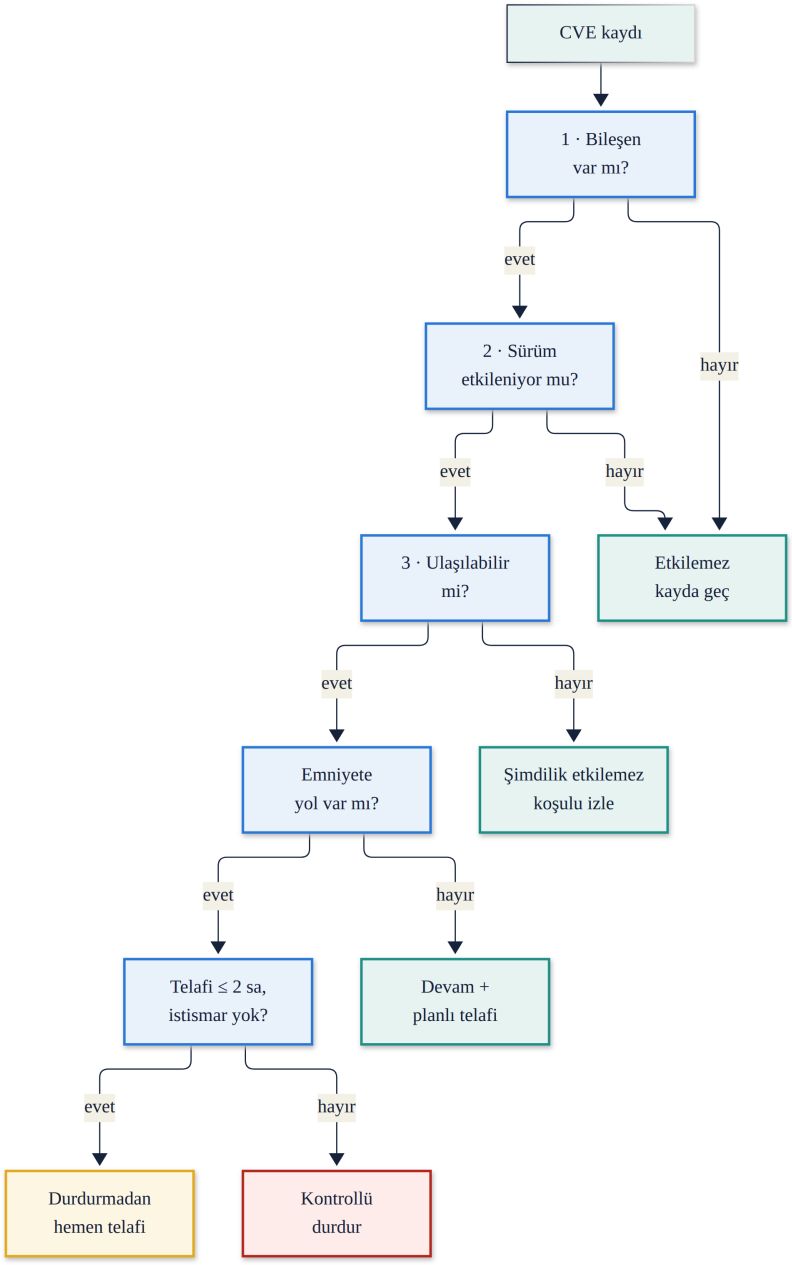
Verilen zafiyet kaydı

Alan	Föy	Efo karşılığı
Bileşen	Component X (PLC firmware kütüphanesi)	Hat PLC'sinin firmware'indeki bir kütüphane (V3, V16)
Sürüm	2.4	2.4
CVE	CVE-XXXX-XXXX	Föydeki temsilî kayıt
Önem	Critical (CVSS 9,8)	CVSS 9,8 tipik olarak ağdan, kimlik doğrulamasız istismar demektir [Y24] SEKTÖR-TİPİK
Yama	Mevcut	2.5 kabul edildi (V17)

SBOM (varsayım). Her bileşen, LAB 4 saldırı grafında bir düğüme bağlandı. Böylece "açık ulaşılabilir mi?" sorusu grafın üzerinde hesaplanabiliyor [Y23] VARSAYIM.

Bileşen	Sürüm	LAB 4 düğümü	Yazılım içerir
PLC firmware (S7-1200 sınıfı)	4.5	PLC	evet
Component X (PLC firmware kütüphanesi)	2.4	PLC	evet
Operatör paneli yazılımı	17.0	HMI	evet
Uzak masaüstü aracı (HMI'da)	15.0	HMI	evet
Bant invertörü firmware + WLAN modülü	3.2	Bant invertörü	evet
4G/VPN router firmware	7.1	4G/VPN router	evet
Emniyet rölesi (kablolu, yazılımsız)	-	Emniyet rölesi / Safety PLC	hayır

Görev — karar akışı



Şekil 1 — Karar akışı. CVSS yalnızca bir girdi; kararı üç şart, emniyete yol ve telafi süresi belirler.

1 Bu CVE sizi gerçekten etkiliyor mu? (üç şartı kontrol edin)

Mevcut (varsayım) mimaride: evet, üç şart da sağlanıyor.

1. **Bileşen var:** Component X, PLC firmware'inin içinde (SBOM) VARSAYIM.
2. **Sürüm etkileniyor:** 2.4 < 2.5 (ilk düzeltilmiş sürüm) VARSAYIM (V17).
3. **Açık ulaşılabilir:** Düz ağda PLC'ye internetten (sürekli açık router), uzak masaüstü aracından, USB'den ve varsayılan parolalı HMI'dan ulaşılıyor (LAB 4.1).

Hedef mimaride: yalnızca içeri girildiği kabul edilirse. Saldırgan VPN ve MFA'yı aşarsa jump server → HMI → PLC yolundan PLC'ye ulaşıyor. Üçüncü şart da ancak bu durumda sağlanıyor.

Karşılaştırma için iki gerçek kayıt aynı akıştan geçirildi:

Kayıt	CVSS	Mimari	Bileşen var	Sürüm etkilenir	Ulaşılabilir	Emniyete yol	Karar
CVE-XXXX-XXXX (Component X)	9,8	Mevcut (varsayım)	✔	✔	✔	✔	Durdurmadan hemen telafi; yama test sonrası

Kayıt	CVSS	Mimari	Bileşen var	Sürüm etkilenir	Ulaşılabilir	Emniyete yol	Karar
CVE-XXXX-XXXX (Component X)	9,8	Hedef, içeri girilmiş kabulü	✔	✔	✔	X	Devam + planlı telafi; yama bakım penceresinde
CVE-2024-47100 (S7-1200)	7,1	Mevcut (varsayım)	✔	✔	✔	✔	Durdurmadan hemen telafi; yama test sonrası
CVE-2024-47100 (S7-1200)	7,1	Hedef, içeri girilmiş kabulü	✔	✔	X	X	Devam: CVE bu hatta etkili değil
SSA-688146 (S7-1500 / ET 200SP)	9,1	Mevcut (varsayım)	X	X	X	—	Devam: CVE bu hatta etkili değil
SSA-688146 (S7-1500 / ET 200SP)	9,1	Hedef, içeri girilmiş kabulü	X	X	X	—	Devam: CVE bu hatta etkili değil

Ulaşılabilirlik, LAB 4'teki giriş noktalarından (internet, uzak araç, USB, yerel HMI, Wi-Fi) bileşenin düğümüne açık bir yol olup olmadığıdır. Hedef sütununda saldırganın VPN ve MFA'yı aştığı kabul edildi.



Grafik aynı tabloyu bir zincir olarak okuyor: bir şart düştüğünde sonrakiler değerlendirilmez. Kırmızı ✔'ler riskin sürdüğü şartlar; kararı, ilk yeşil X'nin nerede durduğu belirliyor ÇIKARIM.

- **CVE-2024-47100** yalnızca PLC web sunucusu açıksa ulaşılabilir. Hedefte web sunucusu kapalı olduğundan sürüm etkilense bile üçüncü şart düşüyor [Y12].
- **SSA-688146**'nın CVSS'i 9,1, ama S7-1500 ve ET 200SP içindir. Bu yüzden ilk şartta eleniyor [Y14]. Yüksek CVSS, kayıt sizi etkilemediği sürece bir şey ifade etmez.

2 Üretimi hemen durdurur musunuz? Neden / neden değil?

Hayır, kendiliğinden durdurmayız. Ama karar mimariye bağlı ve bir durdurma koşulu var.

- **CVSS kararı vermez.** CVSS, açığın genel ciddiyetini ölçer; bu hatta neye yol açacağını söylemez [Y24]. Aynı 9,8'lik kayıt, mevcut ve hedef mimaride farklı karar alıyor (test: `test_cvss_tek_basina_karari_belirlemez`).
- **Mevcut mimaride:** PLC'den emniyet fonksiyonuna yol var (LAB 4: acil stop PLC mantığında). Yani bu açık, emniyeti bozabilen bir açık. Yine de dört telafi edici önlem 1,5 saatte açığı ulaşılamaz kılıyor (Q5). Karar: **durdurmadan hemen telafi**.

- **Durdurma koşulu:** Aktif istismar belirtisi varsa hat **kontrollü durdurulur**. Örnekler: kayıt CISA KEV kataloğunda [Y25], PLC CRC'si referanstan farklı (LAB 6), ya da telafi 2 saatte bitirilemiyor. Bu kural kodda ve testte var (`test_bagimli_emniyette_istismar_belirtisi_varsa_hat_kontrollu_durur`).
- **Hedef mimaride:** PLC ele geçirilse bile emniyet fonksiyonuna yol yok (LAB 4: 0 yol). Açık bir **kullanılabilirlik** riski, emniyet riski değil. Karar: **devam + planlı telafi**, yama bakım penceresinde. Kayıt KEV'e girerse telafi öne çekilir ama hat yine durmaz (`test_bagimsiz_emniyette_aktif_istismar_telafiyi_one_ceker_ama_hatti_durdurmaz`).

Karar bir kez verilip kapanmaz. Aynı kayıt zaman içinde yeni bilgi alır: KEV'e girer, telafi uygulanır, yama gelir. Model, kararı her olayda üç şart ve emniyete yol üzerinden yeniden hesaplıyor VARSAYIM (V23):



#	Olay	Mevcut: karar	Mevcut: durum	Hedef: karar	Hedef: durum
1	Kayıt yayımlandı	**Hemen telafi**	üç şart sağlanıyor	**Devam + planlı telafi**	üç şart sağlanıyor
2	KEV kataloğuna girdi	**Kontrollü durdur**	üç şart sağlanıyor	**Hemen telafi**	üç şart sağlanıyor
3	Telafi uygulandı	**Devam**	3. şart düştü (ulaşılamaz)	**Devam**	3. şart düştü (ulaşılamaz)
4	Yama test edilip uygulandı	**Devam**	2. şart düştü (sürüm etkilenmiyor); SF1–SF4 yeniden doğrulanır	**Devam**	2. şart düştü (sürüm etkilenmiyor)

Telafi, LAB 5.2'deki en kısa önlem kümesidir. Yama, kaydın ilk düzeltilmiş sürümüne (Component X 2.5) geçiştir; mevcut mimaride PLC'den emniyet fonksiyonuna yol olduğu için yamadan sonra emniyet fonksiyonları yeniden doğrulanır.

İki mimari aynı noktada bitiyor ama oraya farklı yoldan geliyor. Mevcut mimaride hat bir kez duruyor ve yamadan sonra emniyet fonksiyonları yeniden doğrulanıyor. Hedef mimaride hat hiç durmuyor ve yama emniyet doğrulamasını tetiklemiyor ÇIKARIM.

3 Yamayı doğrudan mı uygularsınız, önce test mi? Nerede test?

Önce test. PLC firmware yaması, hattı çalıştıran yazılımı değiştirir. Doğrudan uygulamak; haberleşme kaybına, proje uyumsuzluğuna ya da hattın açılmamasına yol açabilir [Y26] SEKTÖR-TİPİK.

- **Nerede:** Aynı firmware ve aynı projeyle kurulmuş **yedek CPU'lu bir test tezgâhında**. Efo'nun atölyesinde FAT düzeni ya da müşteri sahasında hattan ayrı bir pano olabilir VARSAYIM.
- **Sıra** (kod: `yama_adimlari`):
  1. Test tezgâhında yamayı uygula.
  2. Normal fonksiyon testleri: start/stop, yongalama interlock'u, haberleşme.
  3. Validasyon etkileniyorsa emniyet testleri: SF1–SF4 + durma süresi ölçümü (Q4).

4. Bakım penceresinde hatta uygula; eski sürümün yedeği hazır olsun.
5. Yeni konfigürasyonun CRC'sini referans olarak kaydet (LAB 6'da bu referansla karşılaştırma yapılıyor).

4 Bu yama bir safety validation'ı etkiliyor mu? Nasıl anlarsınız?

Mevcut mimaride evet, hedef mimaride hayır. Anlamak için dört kontrol:

1. **Bileşen emniyet zincirinde mi?** SBOM'da emniyet rölesi ya da Safety PLC'nin firmware'i değişiyorsa validasyon her zaman etkilenir (`test_emniyet_rolesi_degisikligi_her_mimaride_validasyon_gerektirir`).
2. **Bileşenden emniyet fonksiyonuna bir bağımlılık yolu var mı?** Bu, LAB 4 grafından hesaplanıyor. Mevcutta PLC → SF1 yolu var: yama, acil stopun davranışını değiştirebilir. Hedefte yol yok: normal PLC'nin yaması emniyeti etkilemez.
3. **Üreticinin sürüm notları:** Yama emniyetle ilgili bir fonksiyonu (STO, SS1, emniyet haberleşmesi) değiştiriyor mu?
4. **Emniyet programının ve parametrelerinin CRC'si:** Yamadan önce ve sonra aynı mı?

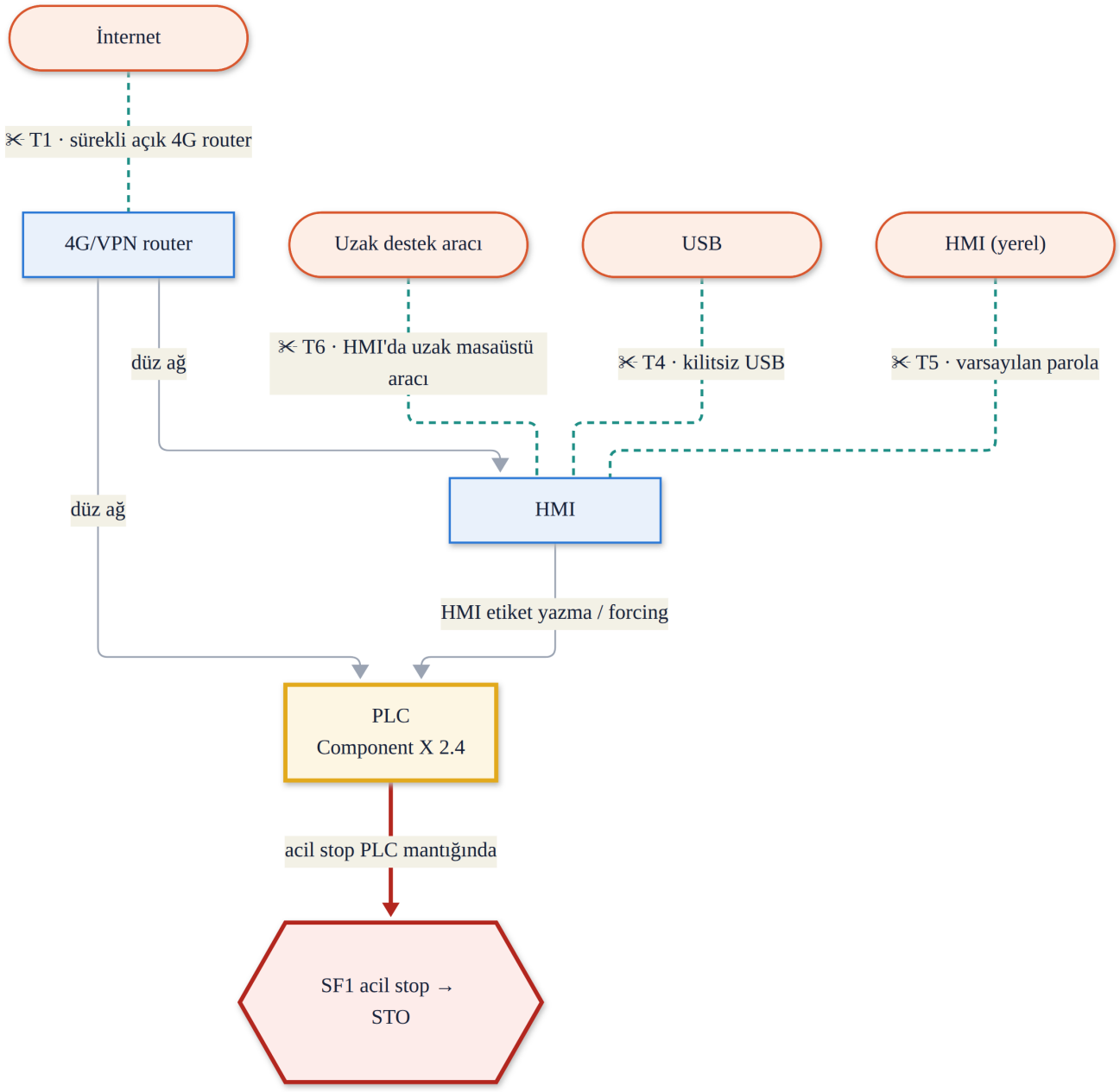
Etkileniyorsa SF1–SF4 fonksiyon testleri ve durma süresi ölçümü tekrarlanır. Bu, LAB 4 QE2'deki bulgunun sayısal karşılığıdır: **bağımsız emniyet, yama döngüsünü hızlandırır**, çünkü her PLC yaması yeniden doğrulama gerektirmez [R61] ÇIKARIM.

5 Yama mümkün değilse compensating control (telafi edici önlem) ne olur?

Telafi edici önlem, açığı kapatmaz; **üçüncü şartı** (ulaşılabilirlik) bozar. Model, açığı ulaşılamaz kılan en kısa süreli önlem kümesini bütün alt kümeleri deneyerek buluyor VARSAYIM (V18):

Mimari	En kısa telafi kümesi	Süre	Telafiden sonra ulaşılabilir mi?	Validasyon etkilenir mi?
Mevcut (varsayım)	T1: Uzak destek router'ının bağlantısını fiziksel olarak kesmek T4: Pano ve HMI USB portlarını kilitlemek T5: HMI varsayılan parolasını değiştirmek T6: HMI'daki uzak masaüstü aracını kaldırmak	1,50 sa	hayır	**evet**
Hedef, içeri girilmiş kabulü	T7: Yama bitene kadar uzak bakım oturumuna (C1) tesis onayı vermemek	0,10 sa	hayır	hayır

Telafi için kabul edilen azami süre: 2,0 saat. Aktif istismar belirtisi varsa ve bileşenden emniyet fonksiyonuna yol varsa karar doğrudan "Kontrollü durdur; telafi ya da yama sonrası yeniden doğrula" olur.



Şekil 2 — Mevcut mimaride telafi planı LAB 4 grafında: yeşil kesikli kenarlar (✂) önlemlerin kestiği girişler. Gri kenarlar açık kalıyor ama artık hiçbir girişten ulaşılamıyor. Kırmızı kenar, PLC ele geçirilirse neyin tehlikeye girdiğini gösteriyor.

- **Mevcutta:** Dört önlem gerekiyor, çünkü PLC'ye dört ayrı giriş noktasından ulaşılıyor. T3 (WLAN) bu kümede yok; Wi-Fi yalnızca invertöre ulaşıyor, PLC'ye değil. Ama invertör parametreleri için ayrıca gerekli (LAB 3, S1).
- **Hedefte:** Tek önlem yetiyor: yama bitene kadar uzak bakım oturumuna onay vermemek. Bu, LAB 4'ün "varsayılan kapalı" ilkesinin doğrudan bir faydası.
- **Ek olarak** SEKTÖR-TİPİK [Y26]:
  - Kullanılmayan servisleri kapatmak (PLC web sunucusu → CVE-2024-47100'ü de kapatır).
  - C3 güvenlik duvarında PLC'ye yalnızca izin listesindeki portları açmak.
  - PLC'ye giden trafiği izlemek ve anormal yazma isteklerinde alarm üretmek.
  - Kaydı KEV kataloğunda izlemek; istismar başlarsa karar "durdur"a döner.

Kanıt: simülasyon (LAB 5)

Eğitim amaçlı modeldir; gerçek bir zafiyet taraması ya da emniyet doğrulaması değildir. SBOM, sürümler, saha değişiklikleri ve EOL kaydı varsayımdır. Mevcut ve hedef mimari varyantları ürün önerisi değildir; mimari farkının karara etkisini göstermek için kuruldu.

Ne gösterildi?

- Üç şart koddan hesaplandı; ulaşılabilirlik LAB 4 saldırı grafından geliyor.
- Karar, CVSS'e göre değil, mimariye göre değişiyor.
- Telafi planı açığı gerçekten ulaşılamaz kılıyor ve 2 saatlik sınırın içinde kalıyor.
- Validasyon ihtiyacı, bileşenden emniyet fonksiyonuna yol olup olmamasıyla belirleniyor.
- Karar her olayda (KEV, telafi, yama) yeniden hesaplanıyor; bağımsız mimaride aktif istismar bile hattı durdurmuyor.
- Güncel olmayan bir SBOM, emniyete yol açan bir açığı "etkilemez" gösterebiliyor (QE2).
- Yaması gelmeyecek bir bileşende telafi kalıcı hâle geliyor (QE3).

Test sonuçları:

#	Test (dosya)	Sonuç
1	`test_surum_karsilastirmasi_sayisaldir` (test_lab5_vuln)	✔ geçti (8/8)
2	`test_foydeki_kayit_mevcut_mimaride_uc_sarti_da_saglar` (test_lab5_vuln)	✔ geçti
3	`test_sbomda_olmayan_urun_cvss_yuksek_olsa_da_etkilemez` (test_lab5_vuln)	✔ geçti
4	`test_web_sunucusu_kapaliysa_csrf_acigi_ulasilamaz` (test_lab5_vuln)	✔ geçti
5	`test_her_sbom_bileseni_saldiri_grafinda_bir_dugume_baglidir` (test_lab5_vuln)	✔ geçti
6	`test_telafi_onlemleri_mimarideki_kosullari_kapatir` (test_lab5_vuln)	✔ geçti
7	`test_cvss_tek_basina_karari_belirlemez` (test_lab5_vuln)	✔ geçti
8	`test_bagimsiz_emniyette_plc_acigi_uretimi_durdurmaz` (test_lab5_vuln)	✔ geçti
9	`test_bagimli_emniyette_istismar_belirtisi_varsa_hat_kontrollu_durur` (test_lab5_vuln)	✔ geçti
10	`test_telafi_plani_acigi_ulasilamaz_kilar_ve_azami_surede_biter` (test_lab5_vuln)	✔ geçti
11	`test_plc_yamasi_validasyonu_yalniz_bagimli_mimaride_etkiler` (test_lab5_vuln)	✔ geçti
12	`test_emniyet_rolesi_degisikligi_her_mimaride_validasyon_gerektirir` (test_lab5_vuln)	✔ geçti
13	`test_yama_once_test_tezgahinda_denenir` (test_lab5_vuln)	✔ geçti (2/2)
14	`test_kev_kaydi_bagimli_emniyette_karari_durdura_cevirir` (test_lab5_vuln)	✔ geçti
15	`test_bagimsiz_emniyette_aktif_istismar_telafiyi_one_ceker_ama_hatti_durdurmaz` (test_lab5_vuln)	✔ geçti
16	`test_telafiden_sonra_acik_ulasilamaz_ve_karar_devama_doner` (test_lab5_vuln)	✔ geçti (2/2)
17	`test_yamadan_sonra_surum_artik_etkilenmez` (test_lab5_vuln)	✔ geçti (2/2)
18	`test_yamadan_sonra_yeniden_dogrulama_yalniz_bagimli_mimaride_gerekir` (test_lab5_vuln)	✔ geçti
19	`test_her_saha_degisikligi_sbom_ile_saha_arasinda_fark_yaratir` (test_lab5_vuln)	✔ geçti
20	`test_eskimis_sbom_emniyete_yol_acan_acigi_etkilemez_gosterir` (test_lab5_vuln)	✔ geçti
21	`test_degisiklik_yonetimine_bagli_sbomda_yanlis_etkilemez_sonucu_sifir` (test_lab5_vuln)	✔ geçti (2/2)
22	`test_eol_kaydi_gercek_bir_sbom_bilesenine_baglidir` (test_lab5_vuln)	✔ geçti
23	`test_yamasi_gelmeyecek_bilesende_telafi_kalici_olur_ve_degisim_planlanir` (test_lab5_vuln)	✔ geçti
24	`test_yamasi_olan_kayitta_kapanis_yamayla_olur` (test_lab5_vuln)	✔ geçti
25	`test_hedef_mimaride_eol_routera_ulasilamaz` (test_lab5_vuln)	✔ geçti

Yorum: En önemli bulgu, LAB 4'teki emniyet bağımsızlığının zafiyet yönetimini de değiştirmesi. Bağımlı mimaride kritik bir PLC açığı hem acil telafi hem de yeniden doğrulama gerektiriyor. Bağımsız mimaride aynı açık, planlı bir bakım işine dönüşüyor.

Ek sorular: zafiyetler ve nasıl kapatılır

> Föy dışıdır; kullanıcı isteğiyle eklendi. Öneriler genel sektör pratiğidir; Efo'nun bunları uygulayıp uygulamadığı bilinmiyor.

E1 CRA, Efo'dan SBOM ve yama konusunda ne istiyor?

- **Durum:** Efo dijital eleman (PLC, HMI, invertör) içeren makineleri AB pazarına arz ediyorsa, CRA'nın üretici yükümlülükleri kapsamına girebilir ÇIKARIM.
- **İstenenler:**
  - Teknik dosyada bir SBOM.
  - Destek süresi boyunca ücretsiz güvenlik güncellemesi; rapor en az 5 yıl diyor [R16].
  - Aktif olarak istismar edilen zafiyetler için 24 sa / 72 sa / 14 gün bildirim; 11.09.2026'dan beri geçerli [Y20, Y22].
- **Kapatma:** Her makine için teslimatta bir SBOM (bileşen, sürüm, tedarikçi). Tedarikçi güvenlik bildirimlerine (Siemens ProductCERT, CISA ICS) abonelik. Müşteriye hangi kurulumun hangi sürümde olduğunu gösteren bir kayıt [Y23] SEKTÖR-TİPİK.

E2 SBOM'un güncel kalmaması bir zafiyet midir?

- **Zafiyet:** Sahada yapılan her firmware güncellemesi, parça değişimi ya da uzak masaüstü aracı kurulumu SBOM'u eskitir. Eski bir SBOM'la yapılan üç şart kontrolü yanlış "etkilemez" sonucu verir ÇIKARIM.
- **Modelde:** İki saha değişikliği kuruldu VARSAYIM (V21). Teslimde PLC yamalıydı (4.7, Component X 2.5). Sonra arızalı CPU, eski firmware'le gelen bir yedekle değişti ve bir teknisyen HMI'ya uzak masaüstü aracı kurdu. Hiçbiri SBOM'a işlenmedi.

Saha değişikliği	Teslim SBOM'unda	Sahada
D1: Arızalı CPU yedek parçayla değişti; yedek CPU eski firmware'le geldi	PLC firmware (S7-1200 sınıfı): 4.7 Component X (PLC firmware kütüphanesi): 2.5	PLC firmware (S7-1200 sınıfı): 4.5 Component X (PLC firmware kütüphanesi): 2.4
D2: Servis teknisyeni HMI'ya uzak masaüstü aracı kurdu	Uzak masaüstü aracı (HMI'da): yok	Uzak masaüstü aracı (HMI'da): 15.0

Kayıt	Mimari	Teslim SBOM'uyla karar	Güncel SBOM'la karar	Yanlış "etkilemez"
CVE-XXXX-XXXX	Mevcut (varsayım)	Devam	Hemen telafi	**evet**
CVE-XXXX-XXXX	Hedef, içeri girilmiş kabulü	Devam	Devam + planlı telafi	**evet**
CVE-2024-47100	Mevcut (varsayım)	Devam	Hemen telafi	**evet**
CVE-2024-47100	Hedef, içeri girilmiş kabulü	Devam	Devam	hayır
SSA-688146	Mevcut (varsayım)	Devam	Devam	hayır
SSA-688146	Hedef, içeri girilmiş kabulü	Devam	Devam	hayır

Yanlış "etkilemez" sonucu: teslim SBOM'uyla **3**, değişiklik yönetimine bağlı SBOM'la **0**. SBOM'da görünmeyen ya da sürümü farklı bileşen sayısı: 3.

- **Sonuç:** Eski SBOM, föydeki kaydı mevcut mimaride "devam" diye geçiriyor. Oysa bileşenden emniyet fonksiyonuna yol var ve doğru karar "hemen telafi" (`test_eskimis_sbom_emniyete_yol_acan_acigi_etkilemez_gosterir`). Yüksek CVSS'li bir kayıt bile, yanlış SBOM'la sessizce elenir.
- **Kapatma:** SBOM'u değişiklik yönetimine bağlamak: her değişiklikte güncellenir ve yeni CRC ile birlikte kaydedilir. Modelde bu durumda yanlış "etkilemez" sayısı sıfır (`test_degisiklik_yonetimine_bagli_sbomda_yanlis_etkilemez_sonucu_sifir`). Etkilemeyen kayıtları gerekçesiyle (ör. "web sunucusu kapalı") kaydetmek. Durum değişirse karar yeniden açılır SEKTÖR-TİPİK [Y23].

E3 Yaması hiç gelmeyecek (ömrü bitmiş) bir bileşen varsa?

- **Zafiyet:** Eski bir HMI işletim sistemi ya da üretimi durmuş bir router için yama gelmez; telafi edici önlem kalıcı hâle gelir.

- **Modelde:** Desteği bitmiş 4G router için temsilî bir kayıt kuruldu; gerçek bir CVE değildir VARSAYIM (V22). Düzeltme hiç gelmeyeceği için bütün sürümler etkilenir.

Kayıt: temsilî kayıt (EOL), 4G router, CVSS 8,8, yama: yok (üretici desteği bitti).

Mimari	Üç şart	Karar	Kapanış	Telafi kümesi
Mevcut (varsayım)	üç şart sağlanıyor	Hemen telafi	**Kalıcı telafi + bileşen değişimi**	T1: Uzak destek router'ının bağlantısını fiziksel olarak kesmek
Hedef, içeri girilmiş kabulü	3. şart düştü (ulaşılamaz)	Devam	Gerekmez (ulaşılamaz)	—

Yama gelmeyecekse telafi geçici değildir: bileşen ömrü boyunca yerinde kalır. Mevcut mimaride bu, uzak desteğin kalıcı olarak kesilmesi demektir; hedef mimaride router zaten emniyet fonksiyonuna giden bir yolun üzerinde değildir.

- **Sonuç:** Mevcut mimaride karar yama penceresine bırakılamaz; telafi (T1) kalıcı olur ve bileşen değiştirme planına girer (`test_yamasi_gelmeyecek_bilesende_telafi_kalici_olur_ve_degisim_planlanir`). Hedef mimaride router hiçbir girişten ulaşılabilir değil, bu yüzden kayıt bu hatta etkili değil (`test_hedef_mimaride_eol_routera_ulasilamaz`).
- **Kapatma:** Bileşeni kendi bölgesine almak (LAB 2) ve yalnızca gerekli akışlara izin vermek. Değiştirme planını bütçelemek. Ömrü biten bileşeni SBOM'da işaretlemek [Y26] SEKTÖR-TİPİK.

Ek — Kod

efo_sim/lab5_vuln.py

```python
"""LAB 5 — SBOM / CVE / yama kararı (eğitim amaçlı).

5.1 Üç şart: bileşen SBOM'da var mı, sürüm etkileniyor mu, açık ulaşılabilir mi?
    Ulaşılabilirlik LAB 4 saldırı grafından hesaplanır (giriş noktalarından o düğüme yol var mı).
5.2 Karar akışı: üretim durur mu, yama nerede test edilir, emniyet validasyonu etkilenir mi,
    yama yoksa hangi telafi edici önlemler açığı kapatır?
5.3 Karar zaman içinde: kayıt KEV'e girer, telafi uygulanır, yama gelir; karar her olayda yeniden hesaplanır.
5.4 SBOM sapması: sahadaki değişiklikler SBOM'a işlenmezse üç şart kontrolü yanlış "etkilemez" der.
5.5 Ömrü bitmiş bileşen: yama hiç gelmeyecekse telafi kalıcı olur ve bileşen değiştirme planına girer.

Simülasyon varyantları ürün önerisi değildir; mimari farkının karara etkisini göstermek içindir.

Çalıştırma (repo kökünden):  python -m efo_sim.lab5_vuln
"""
from itertools import combinations

from efo_sim import lab4_attack as lab4
from efo_sim import stil
from efo_sim.params import OUT_DIR, bolum_metni, d, sayi

MIMARILER = {
    "mevcut": ("Mevcut (varsayım)", lab4.MEVCUT | {"plc_web_sunucusu_acik": d("lab5.plc_web_sunucusu")["mevcut"]}
),
    "hedef": ("Hedef, içeri girilmiş kabulü",
              lab4.HEDEF_ICERI_GIRILDI | {"plc_web_sunucusu_acik": d("lab5.plc_web_sunucusu")["hedef"]}),
}
KARARLAR = {
    "devam": "Devam: CVE bu hatta etkili değil",
    "devam_telafi": "Devam + planlı telafi; yama bakım penceresinde",
    "hemen_telafi": "Durdurmadan hemen telafi; yama test sonrası",
    "durdur": "Kontrollü durdur; telafi ya da yama sonrası yeniden doğrula",
}
KARAR_KISA = {"devam": "Devam", "devam_telafi": "Devam + planlı telafi",
              "hemen_telafi": "Hemen telafi", "durdur": "Kontrollü durdur"}
KARAR_TONU = {"devam": "iyi", "devam_telafi": "bilgi", "hemen_telafi": "uyari", "durdur": "kritik"}
YAMA_YOK = "yok"  # ilk_duzeltilmis alanında: düzeltme yayımlanmayacak (EOL)


def sbom():
    """Sahadaki gerçek SBOM (V16)."""
    return {b["id"]: dict(b) for b in d("lab5.sbom")}


def kayit(kayit_id):
```

```python
    return next(k for k in d("lab5.kayitlar") + [d("lab5.eol_kaydi")] if k["id"] == kayit_id)


def surum(metin):
    return tuple(int(p) for p in metin.split("."))


def surum_etkileniyor(mevcut, ilk_duzeltilmis):
    """İlk düzeltilmiş sürümden önceki her sürüm etkilenir. Düzeltme hiç gelmeyecekse ("yok") hepsi."""
    if mevcut == "-" or ilk_duzeltilmis == "-":
        return False
    if ilk_duzeltilmis == YAMA_YOK:
        return True
    return surum(mevcut) < surum(ilk_duzeltilmis)


# ---------------------------------------------------------------- 5.1 Üç şart
def bilesen(k, sbom_=None):
    """Kayıttaki ürün SBOM'daki hangi bileşen? Yoksa None."""
    return next((b for b in (sbom_ or sbom()).values() if b["urun"] == k["urun"]), None)


def uc_sart(k, mimari, sbom_=None):
    b = bilesen(k, sbom_)
    var = b is not None
    etkilenir = var and surum_etkileniyor(b["surum"], k["ilk_duzeltilmis"])
    ulasilir = var and b["dugum"] in lab4.ulasilabilir(mimari) and all(mimari[c] for c in k["ek_kosullar"])
    return {"var": var, "surum": etkilenir, "ulasilabilir": ulasilir}


def etkiler(k, mimari, sbom_=None):
    return all(uc_sart(k, mimari, sbom_).values())


# ---------------------------------------------------------------- 5.2 Karar akışı
def emniyete_yol(dugum, mimari):
    """Bu düğüm ele geçirilirse emniyet fonksiyonuna (LAB 4 grafı) yol var mı?"""
    return dugum == lab4.HEDEF_DUGUM or bool(lab4.yollar(mimari, dugum))


def validasyon_etkilenir(b, mimari):
    """Yama emniyet fonksiyonunun davranışını değiştirebilir mi? Evet ise yeniden doğrulama gerekir."""
    return b["dugum"] == "emniyet" or emniyete_yol(b["dugum"], mimari)


def telafi_plani(k, mimari, sbom_=None):
    """Açığı ulaşılamaz kılan en kısa süreli önlem kümesi (bütün alt kümeler denenir)."""
    onlemler = [t for t in d("lab5.telafi_onlemleri") if mimari.get(t["kosul"])]
    adaylar = []
    for n in range(1, len(onlemler) + 1):
        for kume in combinations(onlemler, n):
            m = mimari | {t["kosul"]: False for t in kume}
            if not etkiler(k, m, sbom_):
                adaylar.append((sum(t["saat"] for t in kume), len(kume), kume))
    if not adaylar:
        return None
    sure, _, kume = min(adaylar, key=lambda a: (a[0], a[1]))
    return {"onlemler": [t["id"] for t in kume], "saat": sure}


def telafili_mimari(k, mimari):
    """Telafi planı uygulanmış mimari (plan yoksa aynı mimari)."""
    plan = telafi_plani(k, mimari)
    if plan is None:
        return mimari
    onlem = {t["id"]: t for t in d("lab5.telafi_onlemleri")}
    return mimari | {onlem[i]["kosul"]: False for i in plan["onlemler"]}


def uretim_karari(k, mimari, istismar_belirtisi=False, sbom_=None):
    if not etkiler(k, mimari, sbom_):
        return "devam"
    b = bilesen(k, sbom_)
    if not emniyete_yol(b["dugum"], mimari):
        # Emniyet riski yok; ama aktif istismar kullanılabilirlik riskini büyütür: telafi öne çekilir.
        return "hemen_telafi" if istismar_belirtisi else "devam_telafi"
    plan = telafi_plani(k, mimari, sbom_)
    if istismar_belirtisi or plan is None or plan["saat"] > d("lab5.telafi_azami_sure"):
```

```
        return "durdur"
    return "hemen_telafi"


def yama_adimlari(k, mimari):
    """Yama doğrudan uygulanmaz: önce test. Validasyon etkileniyorsa emniyet testleri eklenir."""
    adimlar = ["Yedek CPU'lu test tezgâhında aynı firmware + proje ile yamayı uygula",
               "Normal fonksiyon testleri (start/stop, interlock, haberleşme)"]
    b = bilesen(k)
    if b and validasyon_etkilenir(b, mimari):
        adimlar.append("Emniyet fonksiyonu testleri: SF1–SF4 + durma süresi ölçümü (LAB 1)")
    adimlar += ["Bakım penceresinde hatta uygula; eski sürümün yedeği hazır",
                "Konfigürasyon CRC'sini yeni referans olarak kaydet"]
    return adimlar


# ---------------------------------------------------------------- 5.3 Karar zaman içinde
def olay_akisi(k, mimari):
    """Her olayda kararı yeniden hesaplar. Telafi ulaşılabilirliği, yama sürümü değiştirir."""
    sonuc = []
    for o in d("lab5.olay_akisi"):
        m = telafili_mimari(k, mimari) if o["telafi"] else mimari
        s = sbom()
        b = bilesen(k, s)
        if o["yama"] and b and k["yama"] and k["ilk_duzeltilmis"] not in ("-", YAMA_YOK):
            s[b["id"]]["surum"] = k["ilk_duzeltilmis"]
        karar = uretim_karari(k, m, istismar_belirtisi=o["istismar"], sbom_=s)
        dogrulama = bool(o["yama"] and b and validasyon_etkilenir(b, mimari))
        sonuc.append({"olay": o, "karar": karar, "sart": uc_sart(k, m, s), "dogrulama": dogrulama})
    return sonuc


# ---------------------------------------------------------------- 5.4 SBOM sapması
def teslim_sbom():
    """Teslimde yazılan SBOM: sahadaki değişiklikler (D1, D2…) buna işlenmedi."""
    s = sbom()
    for deg in d("lab5.saha_degisiklikleri"):
        for bid, surum_ in deg["farklar"].items():
            if surum_ is None:
                s.pop(bid, None)
            else:
                s[bid]["surum"] = surum_
    return s


def sbom_farki(eski, gercek):
    """Kayıtlı SBOM ile sahadaki gerçek durum arasındaki farklı bileşenler."""
    return sorted(bid for bid in gercek if bid not in eski or eski[bid]["surum"] != gercek[bid]["surum"])


def yanlis_negatifler(kayitli_sbom, mimari):
    """Gerçekte etkileyen ama kayıtlı SBOM'a göre 'etkilemez' görünen kayıtlar."""
    return [k["id"] for k in d("lab5.kayitlar")
            if etkiler(k, mimari) and not etkiler(k, mimari, kayitli_sbom)]


def degisiklik_yonetimli_sbom():
    """Her saha değişikliği SBOM'a işlenir: kayıtlı SBOM sahadaki durumla aynıdır."""
    return sbom()


# ---------------------------------------------------------------- 5.5 Ömrü bitmiş bileşen
def kapanis(k, mimari):
    """Açık nasıl kapanır? Yama varsa yamayla; yoksa telafi kalıcı olur ve bileşen değiştirilir."""
    if not etkiler(k, mimari):
        return {"yontem": "gerekmez", "plan": None, "degistir": False}
    plan = telafi_plani(k, mimari)
    if k["ilk_duzeltilmis"] == YAMA_YOK:
        return {"yontem": "kalici_telafi", "plan": plan, "degistir": True}
    return {"yontem": "yama", "plan": plan, "degistir": False}


# ---------------------------------------------------------------- Çıktılar: tablolar
def _evet(x):
    return "✓" if x else "X"


def sbom_md():
```

```
    s = ["| Bileşen | Sürüm | LAB 4 düğümü | Yazılım içerir |", "|---|---|---|---|"]
    for b in sbom().values():
        s.append(f"| {b['ad']} | {b['surum']} | {lab4.DUGUMLER[b['dugum']]} | {'evet' if b['yazilim'] else 'hayır
'} |")
    return "\n".join(s) + "\n"


def karar_md():
    s = ["| Kayıt | CVSS | Mimari | Bileşen var | Sürüm etkilenir | Ulaşılabilir | Emniyete yol | Karar |",
         "|---|---|---|---|---|---|---|---|"]
    for k in d("lab5.kayitlar"):
        b = bilesen(k)
        for anahtar, (ad, m) in MIMARILER.items():
            u = uc_sart(k, m)
            yol = _evet(emniyete_yol(b["dugum"], m)) if b else "—"
            s.append(f"| {k['cve']} ({k['urun']}) | {sayi(k['cvss'], 1)} | {ad} | {_evet(u['var'])} | "
                     f"{_evet(u['surum'])} | {_evet(u['ulasilabilir'])} | {yol} | {KARARLAR[uretim_karari(k,
m)]} |")
    s += ["", "Ulaşılabilirlik, LAB 4'teki giriş noktalarından (internet, uzak araç, USB, yerel HMI, Wi-Fi) "
              "bileşenin düğümüne açık bir yol olup olmadığıdır. Hedef sütununda saldırganın VPN ve MFA'yı "
              "aştığı kabul edildi."]
    return "\n".join(s) + "\n"


def telafi_md():
    onlem = {t["id"]: t for t in d("lab5.telafi_onlemleri")}
    k = kayit("FOY-X")
    s = ["| Mimari | En kısa telafi kümesi | Süre | Telafiden sonra ulaşılabilir mi? | Validasyon etkilenir mi?
|",
         "|---|---|---|---|---|"]
    for ad, m in MIMARILER.values():
        p = telafi_plani(k, m)
        if p is None:
            s.append(f"| {ad} | bulunamadı | — | evet | — |")
            continue
        kume = "<br>".join(f"{i}: {onlem[i]['ad']}" for i in p["onlemler"])
        s.append(f"| {ad} | {kume} | {sayi(p['saat'])} sa | {'evet' if etkiler(k, telafili_mimari(k, m)) else 'h
ayır'} | "
                 f"{'**evet**' if validasyon_etkilenir(bilesen(k), m) else 'hayır'} |")
    s += ["", f"Telafi için kabul edilen azami süre: {sayi(d('lab5.telafi_azami_sure'), 1)} saat. "
              "Aktif istismar belirtisi varsa ve bileşenden emniyet fonksiyonuna yol varsa karar doğrudan "
              f"\"{KARARLAR['durdur']}\" olur."]
    return "\n".join(s) + "\n"


def _sart_ozeti(u):
    if not u["var"]:
        return "1. şart düştü (bileşen yok)"
    if not u["surum"]:
        return "2. şart düştü (sürüm etkilenmiyor)"
    if not u["ulasilabilir"]:
        return "3. şart düştü (ulaşılamaz)"
    return "üç şart sağlanıyor"


def zaman_md():
    k = kayit("FOY-X")
    akislar = {a: olay_akisi(k, m) for a, (_, m) in MIMARILER.items()}
    s = ["| # | Olay | Mevcut: karar | Mevcut: durum | Hedef: karar | Hedef: durum |", "|---|---|---|---|---|---|
"]
    for i, o in enumerate(d("lab5.olay_akisi")):
        satir = [str(i + 1), o["ad"]]
        for a in MIMARILER:
            r = akislar[a][i]
            durum = _sart_ozeti(r["sart"]) + ("; SF1-SF4 yeniden doğrulanır" if r["dogrulama"] else "")
            satir += [f"**{KARAR_KISA[r['karar']]}**", durum]
        s.append("| " + " | ".join(satir) + " |")
    s += ["", "Telafi, LAB 5.2'deki en kısa önlem kümesidir. Yama, kaydın ilk düzeltilmiş sürümüne "
              "(Component X 2.5) geçiştir; mevcut mimaride PLC'den emniyet fonksiyonuna yol olduğu için "
              "yamadan sonra emniyet fonksiyonları yeniden doğrulanır."]
    return "\n".join(s) + "\n"


def sapma_md():
    gercek, teslim = sbom(), teslim_sbom()
    s = ["| Saha değişikliği | Teslim SBOM'unda | Sahada |", "|---|---|---|"]
    for deg in d("lab5.saha_degisiklikleri"):
        eski = "<br>".join(f"{gercek[b]['ad']}: {v if v is not None else 'yok'}" for b, v in
```

```
deg["farklar"].items())
        yeni = "<br>".join(f"{gercek[b]['ad']}: {gercek[b]['surum']}" for b in deg["farklar"])
        s.append(f"| {deg['id']}: {deg['ad']} | {eski} | {yeni} |")
    s += ["", "| Kayıt | Mimari | Teslim SBOM'uyla karar | Güncel SBOM'la karar | Yanlış \"etkilemez\" |",
          "|---|---|---|---|---|"]
    for k in d("lab5.kayitlar"):
        for ad, m in MIMARILER.values():
            k_eski, k_yeni = uretim_karari(k, m, sbom_=teslim), uretim_karari(k, m)
            yanlis = k["id"] in yanlis_negatifler(teslim, m)
            s.append(f"| {k['cve']} | {ad} | {KARAR_KISA[k_eski]} | {KARAR_KISA[k_yeni]} | "
                     f"{'**evet**' if yanlis else 'hayır'} |")
    toplam_eski = sum(len(yanlis_negatifler(teslim, m)) for _, m in MIMARILER.values())
    toplam_yeni = sum(len(yanlis_negatifler(degisiklik_yonetimli_sbom(), m)) for _, m in MIMARILER.values())
    s += ["", f"Yanlış \"etkilemez\" sonucu: teslim SBOM'uyla **{toplam_eski}**, değişiklik yönetimine bağlı "
              f"SBOM'la **{toplam_yeni}**. SBOM'da görünmeyen ya da sürümü farklı bileşen sayısı: "
              f"{len(sbom_farki(teslim, gercek))}."]
    return "\n".join(s) + "\n"


def eol_md():
    k = d("lab5.eol_kaydi")
    onlem = {t["id"]: t for t in d("lab5.telafi_onlemleri")}
    yontem = {"gerekmez": "Gerekmez (ulaşılamaz)", "kalici_telafi": "**Kalıcı telafi + bileşen değişimi**",
              "yama": "Yama"}
    s = [f"Kayıt: {k['cve']}, {k['urun']}, CVSS {sayi(k['cvss'], 1)}, yama: {k['yama']}.", "",
         "| Mimari | Üç şart | Karar | Kapanış | Telafi kümesi |", "|---|---|---|---|---|"]
    for ad, m in MIMARILER.values():
        u = uc_sart(k, m)
        kp = kapanis(k, m)
        kume = ("<br>".join(f"{i}: {onlem[i]['ad']}" for i in kp["plan"]["onlemler"])
                if kp["plan"] else "—")
        s.append(f"| {ad} | {_sart_ozeti(u)} | {KARAR_KISA[uretim_karari(k, m)]} | {yontem[kp['yontem']]} |
{kume} |")
    s += ["", "Yama gelmeyecekse telafi geçici değildir: bileşen ömrü boyunca yerinde kalır. Mevcut mimaride "
              "bu, uzak desteğin kalıcı olarak kesilmesi demektir; hedef mimaride router zaten emniyet "
              "fonksiyonuna giden bir yolun üzerinde değildir."]
    return "\n".join(s) + "\n"


# ---------------------------------------------------------------- Çıktılar: diyagram
def mermaid_telafi(k, mimari):
    """Bileşen düğümüne giden yollar ve telafi planının kestiği kenarlar (LAB 4 grafı üzerinde)."""
    b = bilesen(k)
    onlem = {t["id"]: t for t in d("lab5.telafi_onlemleri")}
    plan = telafi_plani(k, mimari)
    kesen = {onlem[i]["kosul"]: i for i in (plan["onlemler"] if plan else [])}
    kenarlar = []
    for g in lab4.GIRISLER:
        for y in lab4.yollar(mimari, g, hedef=b["dugum"]):
            for kn in y:
                if kn not in kenarlar:
                    kenarlar.append(kn)
    emniyet = [kn for kn in lab4.acik_kenarlar(mimari) if kn[0] == b["dugum"] and kn[1] == lab4.HEDEF_DUGUM]
    dugumler = []
    for kn in kenarlar + emniyet:
        for n in kn[:2]:
            if n not in dugumler:
                dugumler.append(n)

    s = ["flowchart TB"]
    for n in dugumler:
        ad = lab4.DUGUMLER[n]
        if n in lab4.GIRISLER:
            s.append(f'  {n}(["{ad}"])')
        elif n == lab4.HEDEF_DUGUM:
            s.append(f'  {n}{{{{"{ad}"}}}}')
        elif n == b["dugum"]:
            s.append(f'  {n}["{ad}<br/>{k["urun"]} {b["surum"]}"]')
        else:
            s.append(f'  {n}["{ad}"]')
    stiller = []
    for i, kn in enumerate(kenarlar + emniyet):
        kes = next((kesen[c] for c in kn[2] if c in kesen), None)
        if kes:
            s.append(f'  {kn[0]} -.-|"✂ {kes} · {kn[3]}"| {kn[1]}')
            stiller.append(stil.kenar_stili(i, "iyi", kesik=True))
        elif kn in emniyet:
            s.append(f'  {kn[0]} ==>|"{kn[3]}"| {kn[1]}')
```

```
            stiller.append(stil.kenar_stili(i, "kritik", kalin=True))
        else:
            s.append(f'  {kn[0]} -->|"{kn[3]}"| {kn[1]}')
            stiller.append(stil.kenar_stili(i, "pasif"))
    s += [stil.classdef("giris", "giris"), stil.classdef("varlik", "bilgi"),
          stil.classdef("bilesen", "uyari", kalin=True), stil.classdef("hedef", "kritik", kalin=True)]
    gir = [n for n in dugumler if n in lab4.GIRISLER]
    var = [n for n in dugumler if n not in lab4.GIRISLER and n not in (b["dugum"], lab4.HEDEF_DUGUM)]
    if gir:
        s.append(f"  class {','.join(gir)} giris")
    if var:
        s.append(f"  class {','.join(var)} varlik")
    s.append(f"  class {b['dugum']} bilesen")
    if emniyet:
        s.append(f"  class {lab4.HEDEF_DUGUM} hedef")
    return "\n".join(s + stiller) + "\n"


# ---------------------------------------------------------------- Çıktılar: grafikler
def _grafik_uc_sart(yol):
    """Kayıt × mimari için şart zinciri: ilk düşen şarttan sonrası değerlendirilmez."""
    plt = stil.mpl()
    from matplotlib.patches import FancyBboxPatch

    sutunlar = ["Bileşen\nvar", "Sürüm\netkilenir", "Ulaşıla-\nbilir", "Emniyete\nyol"]
    satirlar = [(k, ad, m) for k in d("lab5.kayitlar") for ad, m in MIMARILER.values()]
    fig, ax = plt.subplots(figsize=(7.4, 0.42 * len(satirlar) + 1.0))
    ax.set_xlim(0, 11.4)
    ax.set_ylim(len(satirlar) - 0.4, -1.1)
    ax.axis("off")

    x_sutun = [4.05 + i * 1.05 for i in range(len(sutunlar))]
    x_karar = x_sutun[-1] + 0.65
    for x, baslik in zip(x_sutun, sutunlar):
        ax.text(x, -0.75, baslik, ha="center", va="center", fontsize=7.2, color=stil.RENK["ikincil"])
    ax.text(x_karar, -0.75, "Karar", ha="left", va="center", fontsize=7.2, color=stil.RENK["ikincil"])

    onceki = None
    for r, (k, ad, m) in enumerate(satirlar):
        if k["id"] != onceki:
            if onceki is not None:
                ax.axhline(r - 0.5, color=stil.RENK["izgara"], lw=0.8)
            ax.text(0.05, r, f"{k['cve']}", fontsize=7.6, fontweight="bold", va="center")
            ax.text(0.05, r + 0.36, f"{k['urun']} · CVSS {sayi(k['cvss'], 1)}", fontsize=6.4,
                    color=stil.RENK["ikincil"], va="center")
            onceki = k["id"]
        ax.text(2.35, r, ad.split(" (")[0].replace(", içeri girilmiş kabulü", "*"), fontsize=7, va="center",
                color=stil.RENK["ikincil"])
        u = uc_sart(k, m)
        b = bilesen(k)
        degerler = [u["var"], u["surum"], u["ulasilabilir"], emniyete_yol(b["dugum"], m) if b else False]
        dustu = False
        for x, v in zip(x_sutun, degerler):
            if dustu:
                ton, metin = "pasif", "–"
            elif v:
                ton, metin = "kritik", "✓"
            else:
                ton, metin, dustu = "iyi", "X", True
            ax.add_patch(FancyBboxPatch((x - 0.36, r - 0.27), 0.72, 0.54, boxstyle="round,pad=0,rounding_size=0.12",
                                        fc=stil.DOLGU[ton], ec=stil.RENK[ton], lw=0.9))
            ax.text(x, r, metin, ha="center", va="center", fontsize=8.5, color=stil.RENK[ton], fontweight="bold")
        karar = uretim_karari(k, m)
        ton = KARAR_TONU[karar]
        ax.add_patch(FancyBboxPatch((x_karar, r - 0.27), 2.75, 0.54, boxstyle="round,pad=0,rounding_size=0.12",
                                    fc=stil.DOLGU[ton], ec=stil.RENK[ton], lw=0.9))
        ax.text(x_karar + 0.12, r, KARAR_KISA[karar], ha="left", va="center", fontsize=7.4,
                color=stil.RENK["murekkep"], fontweight="bold")
    ax.set_title("Üç şart zinciri: CVSS değil, ilk düşen şart kararı belirler", pad=4)
    ax.text(0.05, len(satirlar) - 0.25, "✓ şart sağlanıyor (risk sürüyor) · X şart düştü · – değerlendirilmez · "
            "* saldırganın VPN + MFA'yı aştığı kabul edildi", fontsize=6.4, color=stil.RENK["ikincil"], va="top")
    fig.savefig(yol)
    plt.close(fig)


def _grafik_zaman(yol):
    """Olay akışında kararın iki mimaride nasıl değiştiği (FOY-X)."""
```

```
    plt = stil.mpl()
    from matplotlib.patches import FancyArrowPatch, FancyBboxPatch

    k = kayit("FOY-X")
    olaylar = d("lab5.olay_akisi")
    n = len(olaylar)
    fig, ax = plt.subplots(figsize=(7.6, 3.05))
    ax.set_xlim(-1.55, n - 0.45)
    ax.set_ylim(2.05, -0.95)
    ax.axis("off")
    for i, o in enumerate(olaylar):
        ax.text(i, -0.62, f"{i + 1}", ha="center", va="center", fontsize=7.5, fontweight="bold", color="white",
                bbox=dict(boxstyle="circle,pad=0.25", fc=stil.RENK["murekkep"], ec="none"))
        ax.text(i, -0.25, o["ad"], ha="center", va="center", fontsize=7, color=stil.RENK["murekkep"])
    for r, (a, (ad, m)) in enumerate(MIMARILER.items()):
        y = r * 1.15 + 0.45
        ax.text(-1.5, y, ad.replace(", içeri girilmiş kabulü", "\n(içeri girilmiş)"), ha="left", va="center",
                fontsize=7.4, fontweight="bold", color=stil.RENK["ikincil"])
        akis = olay_akisi(k, m)
        for i, sonuc in enumerate(akis):
            ton = KARAR_TONU[sonuc["karar"]]
            ax.add_patch(FancyBboxPatch((i - 0.44, y - 0.3), 0.88, 0.6,
boxstyle="round,pad=0,rounding_size=0.09",
                                        fc=stil.DOLGU[ton], ec=stil.RENK[ton], lw=1.1))
            ax.text(i, y - 0.06, KARAR_KISA[sonuc["karar"]].replace(" + ", " +\n"), ha="center", va="center",
                    fontsize=7, fontweight="bold", color=stil.RENK["murekkep"], linespacing=1.05)
            alt = "yeniden doğrula" if sonuc["dogrulama"] else _sart_ozeti(sonuc["sart"]).split(" (")[0]
            ax.text(i, y + 0.2, alt, ha="center", va="center", fontsize=5.6, color=stil.RENK["ikincil"])
            if i:
                ax.add_patch(FancyArrowPatch((i - 0.56, y), (i - 0.44, y), arrowstyle="-|>", mutation_scale=7,
                                             color=stil.RENK["cizgi"], lw=1))
    ax.set_title("Karar kapanmaz: her olayda yeniden hesaplanır (föy kaydı, CVSS 9,8)", pad=2)
    fig.savefig(yol)
    plt.close(fig)


def main():
    OUT_DIR.mkdir(exist_ok=True)
    yaz = {
        "lab5_sbom.md": sbom_md(),
        "lab5_karar.md": karar_md(),
        "lab5_telafi.md": telafi_md(),
        "lab5_zaman.md": zaman_md(),
        "lab5_sapma.md": sapma_md(),
        "lab5_eol.md": eol_md(),
        "lab5_telafi_mevcut.mmd": mermaid_telafi(kayit("FOY-X"), MIMARILER["mevcut"][1]),
        "lab5_params.yaml": bolum_metni("lab5"),
    }
    for ad, metin in yaz.items():
        (OUT_DIR / ad).write_text(metin, encoding="utf-8")
    _grafik_uc_sart(OUT_DIR / "lab5_uc_sart.png")
    _grafik_zaman(OUT_DIR / "lab5_karar_zaman.png")
    print(karar_md())
    print(zaman_md())
    print(sapma_md())
    print(eol_md())


if __name__ == "__main__":
    main()
```

efo_sim/tests/test_lab5_vuln.py

```
"""LAB 5 gereksinimleri = testler (eğitim amaçlı model)."""
import pytest

from efo_sim import lab5_vuln as m

MEVCUT = m.MIMARILER["mevcut"][1]
HEDEF = m.MIMARILER["hedef"][1]


# ---------------------------------------------------------------- 5.1 Üç şart
@pytest.mark.parametrize("mevcut,duzeltilmis,beklenen", [
    ("2.4", "2.5", True), ("2.5", "2.5", False), ("2.10", "2.5", False),
    ("4.5", "4.7", True), ("4.7", "4.7", False), ("-", "2.5", False),
    ("7.1", "yok", True), ("-", "yok", False),
])
```

```python
def test_surum_karsilastirmasi_sayisaldir(mevcut, duzeltilmis, beklenen):
    assert m.surum_etkileniyor(mevcut, duzeltilmis) is beklenen


def test_foydeki_kayit_mevcut_mimaride_uc_sarti_da_saglar():
    assert m.uc_sart(m.kayit("FOY-X"), MEVCUT) == {"var": True, "surum": True, "ulasilabilir": True}


def test_sbomda_olmayan_urun_cvss_yuksek_olsa_da_etkilemez():
    k = m.kayit("SSA-688146")
    assert k["cvss"] >= 9.0
    assert not m.uc_sart(k, MEVCUT)["var"]
    assert m.uretim_karari(k, MEVCUT) == "devam"


def test_web_sunucusu_kapaliysa_csrf_acigi_ulasilamaz():
    k = m.kayit("CVE-2024-47100")
    assert m.etkiler(k, MEVCUT)
    assert m.uc_sart(k, HEDEF)["surum"] and not m.uc_sart(k, HEDEF)["ulasilabilir"]


def test_her_sbom_bileseni_saldiri_grafinda_bir_dugume_baglidir():
    for b in m.sbom().values():
        assert b["dugum"] in m.lab4.DUGUMLER


def test_telafi_onlemleri_mimarideki_kosullari_kapatir():
    for t in m.d("lab5.telafi_onlemleri"):
        assert t["kosul"] in MEVCUT and t["kosul"] in HEDEF


# ---------------------------------------------------------------- 5.2 Karar akışı
def test_cvss_tek_basina_karari_belirlemez():
    """Aynı 9,8'lik kayıt mimariye göre farklı karar alır."""
    k = m.kayit("FOY-X")
    assert m.uretim_karari(k, MEVCUT) != m.uretim_karari(k, HEDEF)


def test_bagimsiz_emniyette_plc_acigi_uretimi_durdurmaz():
    assert m.uretim_karari(m.kayit("FOY-X"), HEDEF) == "devam_telafi"


def test_bagimli_emniyette_istismar_belirtisi_varsa_hat_kontrollu_durur():
    assert m.uretim_karari(m.kayit("FOY-X"), MEVCUT, istismar_belirtisi=True) == "durdur"


def test_telafi_plani_acigi_ulasilamaz_kilar_ve_azami_surede_biter():
    k = m.kayit("FOY-X")
    onlem = {t["id"]: t for t in m.d("lab5.telafi_onlemleri")}
    for _, mim in m.MIMARILER.values():
        p = m.telafi_plani(k, mim)
        assert p is not None and p["saat"] <= m.d("lab5.telafi_azami_sure")
        assert not m.etkiler(k, mim | {onlem[i]["kosul"]: False for i in p["onlemler"]})


def test_plc_yamasi_validasyonu_yalniz_bagimli_mimaride_etkiler():
    plc = m.sbom()["PLC_FW"]
    assert m.validasyon_etkilenir(plc, MEVCUT)
    assert not m.validasyon_etkilenir(plc, HEDEF)


def test_emniyet_rolesi_degisikligi_her_mimaride_validasyon_gerektirir():
    role = m.sbom()["EMNIYET_RELE"]
    assert m.validasyon_etkilenir(role, MEVCUT) and m.validasyon_etkilenir(role, HEDEF)


@pytest.mark.parametrize("mimari", ["mevcut", "hedef"])
def test_yama_once_test_tezgahinda_denenir(mimari):
    adimlar = m.yama_adimlari(m.kayit("FOY-X"), m.MIMARILER[mimari][1])
    assert "test tezgâhı" in adimlar[0]
    emniyet_testi = any("SF1" in a for a in adimlar)
    assert emniyet_testi is (mimari == "mevcut")


# ---------------------------------------------------------------- 5.3 Karar zaman içinde
def _akis(mimari):
    return {r["olay"]["id"]: r for r in m.olay_akisi(m.kayit("FOY-X"), m.MIMARILER[mimari][1])}

```

```python

def test_kev_kaydi_bagimli_emniyette_karari_durdura_cevirir():
    a = _akis("mevcut")
    assert a["yayim"]["karar"] == "hemen_telafi"
    assert a["kev"]["karar"] == "durdur"


def test_bagimsiz_emniyette_aktif_istismar_telafiyi_one_ceker_ama_hatti_durdurmaz():
    a = _akis("hedef")
    assert a["yayim"]["karar"] == "devam_telafi"
    assert a["kev"]["karar"] == "hemen_telafi"
    assert all(r["karar"] != "durdur" for r in a.values())


@pytest.mark.parametrize("mimari", ["mevcut", "hedef"])
def test_telafiden_sonra_acik_ulasilamaz_ve_karar_devama_doner(mimari):
    r = _akis(mimari)["telafi"]
    assert not r["sart"]["ulasilabilir"] and r["karar"] == "devam"


@pytest.mark.parametrize("mimari", ["mevcut", "hedef"])
def test_yamadan_sonra_surum_artik_etkilenmez(mimari):
    r = _akis(mimari)["yama"]
    assert not r["sart"]["surum"] and r["karar"] == "devam"


def test_yamadan_sonra_yeniden_dogrulama_yalniz_bagimli_mimaride_gerekir():
    assert _akis("mevcut")["yama"]["dogrulama"]
    assert not _akis("hedef")["yama"]["dogrulama"]


# ---------------------------------------------------------------- 5.4 SBOM sapması
def test_her_saha_degisikligi_sbom_ile_saha_arasinda_fark_yaratir():
    teslim, gercek = m.teslim_sbom(), m.sbom()
    fark = set(m.sbom_farki(teslim, gercek))
    for deg in m.d("lab5.saha_degisiklikleri"):
        assert set(deg["farklar"]) <= fark


def test_eskimis_sbom_emniyete_yol_acan_acigi_etkilemez_gosterir():
    """Yedek CPU eski firmware'le geldi; teslim SBOM'u buna bakıp 'devam' diyor."""
    k = m.kayit("FOY-X")
    assert "FOY-X" in m.yanlis_negatifler(m.teslim_sbom(), MEVCUT)
    assert m.uretim_karari(k, MEVCUT, sbom_=m.teslim_sbom()) == "devam"
    assert m.emniyete_yol(m.bilesen(k)["dugum"], MEVCUT)


@pytest.mark.parametrize("mimari", ["mevcut", "hedef"])
def test_degisiklik_yonetimine_bagli_sbomda_yanlis_etkilemez_sonucu_sifir(mimari):
    assert m.yanlis_negatifler(m.degisiklik_yonetimli_sbom(), m.MIMARILER[mimari][1]) == []


# ---------------------------------------------------------------- 5.5 Ömrü bitmiş bileşen
def test_eol_kaydi_gercek_bir_sbom_bilesenine_baglidir():
    assert m.bilesen(m.d("lab5.eol_kaydi")) is not None


def test_yamasi_gelmeyecek_bilesende_telafi_kalici_olur_ve_degisim_planlanir():
    kp = m.kapanis(m.d("lab5.eol_kaydi"), MEVCUT)
    assert kp["yontem"] == "kalici_telafi" and kp["degistir"]
    assert kp["plan"] is not None and kp["plan"]["saat"] <= m.d("lab5.telafi_azami_sure")


def test_yamasi_olan_kayitta_kapanis_yamayla_olur():
    assert m.kapanis(m.kayit("FOY-X"), MEVCUT)["yontem"] == "yama"


def test_hedef_mimaride_eol_routera_ulasilamaz():
    k = m.d("lab5.eol_kaydi")
    assert not m.etkiler(k, HEDEF)
    assert m.kapanis(k, HEDEF)["yontem"] == "gerekmez"
```

`params.yaml` — LAB 5 bölümü:

efo_sim/out/lab5_params.yaml

```yaml
lab5:
  # SBOM: hat yazılım bileşenleri. "dugum" = LAB 4 saldırı grafındaki karşılığı
```

```yaml
  sbom:
    deger:
      - {id: PLC_FW, ad: "PLC firmware (S7-1200 sınıfı)", urun: "S7-1200", dugum: plc, surum: "4.5", yazilim: true}
      - {id: COMP_X, ad: "Component X (PLC firmware kütüphanesi)", urun: "Component X", dugum: plc, surum: "2.4", yazilim: true}
      - {id: HMI_SW, ad: "Operatör paneli yazılımı", urun: "HMI runtime", dugum: hmi, surum: "17.0", yazilim: true}
      - {id: UZAK_ARAC, ad: "Uzak masaüstü aracı (HMI'da)", urun: "Uzak masaüstü", dugum: hmi, surum: "15.0", yazilim: true}
      - {id: VFD_FW, ad: "Bant invertörü firmware + WLAN modülü", urun: "VFD", dugum: vfd, surum: "3.2", yazilim: true}
      - {id: ROUTER_FW, ad: "4G/VPN router firmware", urun: "4G router", dugum: router, surum: "7.1", yazilim: true}
      - {id: EMNIYET_RELE, ad: "Emniyet rölesi (kablolu, yazılımsız)", urun: "Emniyet rölesi", dugum: emniyet, surum: "-", yazilim: false}
    etiket: "VARSAYIM"
    kaynak: "V2, V3, V5, V16"
    not: "Efo SBOM yayınlamıyor. Bileşenler LAB 2 varlık listesinden; sürümler kabul. Component X ve 2.4 föyden"

  # Zafiyet kayıtları: föydeki kayıt + iki gerçek kayıt (karşılaştırma için)
  kayitlar:
    deger:
      - {id: FOY-X, cve: "CVE-XXXX-XXXX", urun: "Component X", ilk_duzeltilmis: "2.5", cvss: 9.8,
         vektor: "ağ (kimlik doğrulamasız)", ek_kosullar: [], yama: "2.5", kaynak: "föy LAB 5 + V17"}
      - {id: CVE-2024-47100, cve: "CVE-2024-47100", urun: "S7-1200", ilk_duzeltilmis: "4.7", cvss: 7.1,
         vektor: "ağ, PLC web arayüzü (CSRF, kullanıcı etkileşimi)", ek_kosullar: [plc_web_sunucusu_acik], yama: "4.7", kaynak: "Y12"}
      - {id: SSA-688146, cve: "SSA-688146", urun: "S7-1500 / ET 200SP", ilk_duzeltilmis: "-", cvss: 9.1,
         vektor: "ağ", ek_kosullar: [], yama: "üretici bildirimine göre", kaynak: "Y14"}
    etiket: "SEKTÖR-TİPİK"
    kaynak: "föy LAB 5, Y12, Y14"
    not: "FOY-X'in etkilenen sürüm aralığı föyde yok; 2.5 öncesi etkilenir kabul edildi (V17)"

  # LAB 4 mimarilerine eklenen koşul
  plc_web_sunucusu:
    deger: {mevcut: true, hedef: false}
    etiket: "VARSAYIM"
    kaynak: "V13"
    not: "Mevcutta PLC web sunucusu açık; hedefte kapalı"

  # Telafi edici önlemler: kapattığı koşul (LAB 4 mimari anahtarı) ve uygulama süresi
  telafi_onlemleri:
    deger:
      - {id: T1, ad: "Uzak destek router'ının bağlantısını fiziksel olarak kesmek", kosul: router_surekli_acik, saat: 0.25}
      - {id: T2, ad: "PLC web sunucusunu kapatmak", kosul: plc_web_sunucusu_acik, saat: 1.0}
      - {id: T3, ad: "İnvertör WLAN modülünü sökmek", kosul: wlan_modulu_acik, saat: 0.25}
      - {id: T4, ad: "Pano ve HMI USB portlarını kilitlemek", kosul: usb_kilitsiz, saat: 0.5}
      - {id: T5, ad: "HMI varsayılan parolasını değiştirmek", kosul: hmi_varsayilan_parola, saat: 0.25}
      - {id: T6, ad: "HMI'daki uzak masaüstü aracını kaldırmak", kosul: uzak_masaustu_kurulu, saat: 0.5}
      - {id: T7, ad: "Yama bitene kadar uzak bakım oturumuna (C1) tesis onayı vermemek", kosul: uzak_oturum_acik, saat: 0.1}
    etiket: "VARSAYIM"
    kaynak: "V18, Y15"
    not: "Süreler bir bakım teknisyeni için kaba tahmin"
  telafi_azami_sure:
    {deger: 2.0, birim: "saat", etiket: "VARSAYIM", kaynak: "V18",
     not: "Emniyete yol açan bir zafiyette telafi bu süre içinde bitirilemiyorsa hat kontrollü durdurulur"}

  # 5.3 Karar bir kez verilip kapanmaz: aynı kayıt için olaylar sırayla gelir, karar her olayda yeniden hesaplanır
  olay_akisi:
    deger:
      - {id: yayim, ad: "Kayıt yayımlandı", istismar: false, telafi: false, yama: false}
      - {id: kev, ad: "KEV kataloğuna girdi", istismar: true, telafi: false, yama: false}
      - {id: telafi, ad: "Telafi uygulandı", istismar: true, telafi: true, yama: false}
      - {id: yama, ad: "Yama test edilip uygulandı", istismar: true, telafi: true, yama: true}
    etiket: "VARSAYIM"
    kaynak: "V23, Y25"
    not: "Sıra tipik bir zafiyet yaşam döngüsü. KEV = CISA'nın aktif istismar edilen açıklar kataloğu"

  # 5.4 SBOM sapması: sahadaki değişiklikler teslim SBOM'una işlenmezse. farklar = teslim SBOM'unda yazan sürüm (null: teslimde yok)
  saha_degisiklikleri:
    deger:
      - {id: D1, ad: "Arızalı CPU yedek parçayla değişti; yedek CPU eski firmware'le geldi",
         farklar: {PLC_FW: "4.7", COMP_X: "2.5"}}
      - {id: D2, ad: "Servis teknisyeni HMI'ya uzak masaüstü aracı kurdu", farklar: {UZAK_ARAC: null}}
```

```
    etiket: "VARSAYIM"
    kaynak: "V21, Y23"
    not: "Teslimde PLC 4.7 / Component X 2.5 (yamalı). Sahadaki gerçek durum lab5.sbom (V16)"

  # 5.5 Ömrü bitmiş bileşen: yaması hiç gelmeyecek temsilî kayıt (ilk_duzeltilmis: "yok" = bütün sürümler
etkilenir)
  eol_kaydi:
    deger: {id: TEMSILI-EOL, cve: "temsilî kayıt (EOL)", urun: "4G router", ilk_duzeltilmis: "yok", cvss: 8.8,
            vektor: "ağ, router yönetim arayüzü", ek_kosullar: [], yama: "yok (üretici desteği bitti)"}
    etiket: "VARSAYIM"
    kaynak: "V22, Y26"
    not: "Gerçek bir CVE değildir; QE3'ü sınamak için kuruldu. Router sürümü lab5.sbom'daki ROUTER_FW"
```

pytest çıktısı:

efo_sim/out/pytest_lab5.txt

```
============================= test session starts ==============================
platform linux -- Python 3.14.8, pytest-9.0.2, pluggy-1.6.0 -- /usr/bin/python
rootdir: /home/ydogan/Masaüstü/safety40-efo-konveyor
configfile: pytest.ini
plugins: asyncio-1.3.0, anyio-4.13.0
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None,
asyncio_default_test_loop_scope=function
collecting ... collected 36 items

efo_sim/tests/test_lab5_vuln.py::test_surum_karsilastirmasi_sayisaldir[2.4-2.5-True] PASSED [  2%]
efo_sim/tests/test_lab5_vuln.py::test_surum_karsilastirmasi_sayisaldir[2.5-2.5-False] PASSED [  5%]
efo_sim/tests/test_lab5_vuln.py::test_surum_karsilastirmasi_sayisaldir[2.10-2.5-False] PASSED [  8%]
efo_sim/tests/test_lab5_vuln.py::test_surum_karsilastirmasi_sayisaldir[4.5-4.7-True] PASSED [ 11%]
efo_sim/tests/test_lab5_vuln.py::test_surum_karsilastirmasi_sayisaldir[4.7-4.7-False] PASSED [ 13%]
efo_sim/tests/test_lab5_vuln.py::test_surum_karsilastirmasi_sayisaldir[--2.5-False] PASSED [ 16%]
efo_sim/tests/test_lab5_vuln.py::test_surum_karsilastirmasi_sayisaldir[7.1-yok-True] PASSED [ 19%]
efo_sim/tests/test_lab5_vuln.py::test_surum_karsilastirmasi_sayisaldir[--yok-False] PASSED [ 22%]
efo_sim/tests/test_lab5_vuln.py::test_foydeki_kayit_mevcut_mimaride_uc_sarti_da_saglar PASSED [ 25%]
efo_sim/tests/test_lab5_vuln.py::test_sbomda_olmayan_urun_cvss_yuksek_olsa_da_etkilemez PASSED [ 27%]
efo_sim/tests/test_lab5_vuln.py::test_web_sunucusu_kapaliysa_csrf_acigi_ulasilamaz PASSED [ 30%]
efo_sim/tests/test_lab5_vuln.py::test_her_sbom_bileseni_saldiri_grafinda_bir_dugume_baglidir PASSED [ 33%]
efo_sim/tests/test_lab5_vuln.py::test_telafi_onlemleri_mimarideki_kosullari_kapatir PASSED [ 36%]
efo_sim/tests/test_lab5_vuln.py::test_cvss_tek_basina_karari_belirlemez PASSED [ 38%]
efo_sim/tests/test_lab5_vuln.py::test_bagimsiz_emniyette_plc_acigi_uretimi_durdurmaz PASSED [ 41%]
efo_sim/tests/test_lab5_vuln.py::test_bagimli_emniyette_istismar_belirtisi_varsa_hat_kontrollu_durur PASSED [ 44%]
efo_sim/tests/test_lab5_vuln.py::test_telafi_plani_acigi_ulasilamaz_kilar_ve_azami_surede_biter PASSED [ 47%]
efo_sim/tests/test_lab5_vuln.py::test_plc_yamasi_validasyonu_yalniz_bagimli_mimaride_etkiler PASSED [ 50%]
efo_sim/tests/test_lab5_vuln.py::test_emniyet_rolesi_degisikligi_her_mimaride_validasyon_gerektirir PASSED [ 52%]
efo_sim/tests/test_lab5_vuln.py::test_yama_once_test_tezgahinda_denenir[mevcut] PASSED [ 55%]
efo_sim/tests/test_lab5_vuln.py::test_yama_once_test_tezgahinda_denenir[hedef] PASSED [ 58%]
efo_sim/tests/test_lab5_vuln.py::test_kev_kaydi_bagimli_emniyette_karari_durdura_cevirir PASSED [ 61%]
efo_sim/tests/test_lab5_vuln.py::test_bagimsiz_emniyette_aktif_istismar_telafiyi_one_ceker_ama_hatti_durdurmaz PASSED
[ 63%]
efo_sim/tests/test_lab5_vuln.py::test_telafiden_sonra_acik_ulasilamaz_ve_karar_devama_doner[mevcut] PASSED [ 66%]
efo_sim/tests/test_lab5_vuln.py::test_telafiden_sonra_acik_ulasilamaz_ve_karar_devama_doner[hedef] PASSED [ 69%]
efo_sim/tests/test_lab5_vuln.py::test_yamadan_sonra_surum_artik_etkilenmez[mevcut] PASSED [ 72%]
efo_sim/tests/test_lab5_vuln.py::test_yamadan_sonra_surum_artik_etkilenmez[hedef] PASSED [ 75%]
efo_sim/tests/test_lab5_vuln.py::test_yamadan_sonra_yeniden_dogrulama_yalniz_bagimli_mimaride_gerekir PASSED [ 77%]
efo_sim/tests/test_lab5_vuln.py::test_her_saha_degisikligi_sbom_ile_saha_arasinda_fark_yaratir PASSED [ 80%]
efo_sim/tests/test_lab5_vuln.py::test_eskimis_sbom_emniyete_yol_acan_acigi_etkilemez_gosterir PASSED [ 83%]
efo_sim/tests/test_lab5_vuln.py::test_degisiklik_yonetimine_bagli_sbomda_yanlis_etkilemez_sonucu_sifir[mevcut] PASSED
[ 86%]
efo_sim/tests/test_lab5_vuln.py::test_degisiklik_yonetimine_bagli_sbomda_yanlis_etkilemez_sonucu_sifir[hedef] PASSED
[ 88%]
efo_sim/tests/test_lab5_vuln.py::test_eol_kaydi_gercek_bir_sbom_bilesenine_baglidir PASSED [ 91%]
efo_sim/tests/test_lab5_vuln.py::test_yamasi_gelmeyecek_bilesende_telafi_kalici_olur_ve_degisim_planlanir PASSED
[ 94%]
efo_sim/tests/test_lab5_vuln.py::test_yamasi_olan_kayitta_kapanis_yamayla_olur PASSED [ 97%]
efo_sim/tests/test_lab5_vuln.py::test_hedef_mimaride_eol_routera_ulasilamaz PASSED [100%]

============================== 36 passed in 0.11s ==============================
```

Kaynaklar

R = derin araştırma raporundaki kaynak numarası; Y = bu çalışmada eklenen kaynak.

- **R16** Knapster — Cyber Resilience Act: Deadlines — https://www.knapster.at/en/wissen/cyber-resilience-act-fristen-produktklassen.html

- **R61** IEC 61508 Functional Safety — SIL, PFD, Safety PLC — https://iotdigitaltwinplm.com/iec-61508-functional-safety-sil-pfd-safety-instrumented-systems-safety-plc-architecture-2026/
- **Y12** CISA ICSA-25-021-02 — Siemens SIMATIC S7-1200 CPUs (CVE-2024-47100) — https://cisa.gov/news-events/ics-advisories/icsa-25-021-02
- **Y14** Siemens ProductCERT SSA-688146 — https://cert-portal.siemens.com/productcert/html/ssa-688146.html
- **Y20** The Cyber Resilience Act Explained — https://www.cyberresilienceact.eu/explained.html
- **Y22** Regulation (EU) 2024/2847 (Cyber Resilience Act), Article 14 — Reporting obligations of manufacturers — https://eur-lex.europa.eu/eli/reg/2024/2847/oj
- **Y23** CISA — Software Bill of Materials (SBOM) — https://www.cisa.gov/sbom
- **Y24** FIRST — CVSS v3.1 Specification Document — https://www.first.org/cvss/v3.1/specification-document
- **Y25** CISA — Known Exploited Vulnerabilities Catalog — https://www.cisa.gov/known-exploited-vulnerabilities-catalog
- **Y26** NIST SP 800-82 Rev. 3 — Guide to Operational Technology (OT) Security — https://csrc.nist.gov/pubs/sp/800/82/r3/final